

TEMSİLCİLER VE OLAYLAR

Temsilciler

Temsilci (**delegate**), yöntemleri dolaylı olarak çağırmanın bir yoludur. Saf nesne yönelimli programlama dillerinde (Java ve C#), yarattığı karmaşadan ötürü **gösterici** (**pointer**) kullanımı tercih edilmez. Bu sebeple **işlev göstericisi** (**function pointer**) yerine, ondan daha yetenekli olan temsilciler kullanılır. Temsilciler, birden fazla **yöntemi** (**method**) referans gösterebilirler.

```
// See https://aka.ms/new-console-template for more information
public delegate void Yap(string iş); // Temsilci tanımı
/*
    Temsilci tanımı:
    public delegate void Yap(string iş);
    - Yap: Temsilci adı
    - string: Temsilciye gönderilecek parametre türü
    - void: Temsilcinin dönüş türü
    Temsilci, bir veya daha fazla yöntemi temsil eden bir türdür.
*/
public class Program
{
    public static void Main(string[] args)
    {
        YöntemliSınıf nesne = new YöntemliSınıf();

        Yap işyap = nesne.MusluğuAç; // tek bir yöntemi dolaylı çağırma

        // İstenirse birden fazla yöntemi temsil edebilir:
        işyap += nesne.MusluğuKapat;
        işyap += YöntemliSınıf.BulaşıkYıkı;

        işyap("İlhan Özkan"); // "Yap" temsilcisi çağrıldı (invoke the delegate)
        /*
            İlhan Özkan Musluğu Açtı.
            İlhan Özkan Musluğu Kapattı.
            İlhan Özkan Bulaşık Yıkadı.
        */
    }
}

public class YöntemliSınıf
{
    public void MusluğuAç(string açmessage) // Temsilci tanımındaki imzaya uygun yöntem
    {
        Console.WriteLine("{0} Musluğu Açtı.", açmessage);
    }
    public void MusluğuKapat(string kapatmessage) // Temsilci tanımındaki imzaya uygun yöntem
    {
        Console.WriteLine("{0} Musluğu Kapattı.", kapatmessage);
    }
    public static void BulaşıkYıkı(string yıkamessage)
    // Temsilci tanımındaki imzaya uygun yöntem ama static tanımlanmış.
    {
        Console.WriteLine("{0} Bulaşık Yıkadı.", yıkamessage);
    }
}
```

Lamda İfadeleri

Bir **lamda ifadesi** (**lambda expression**), yöntemleri dolaylı oluşturmak için özlü bir yol sağlar. ‘Lamda’ adı, *Alonzo Church* tarafından 1930'larda mantık ve hesaplanabilme hakkındaki soruları araştırmak

için icat edilen matematiksel bir form olan lamda hesabından gelir. Lamda hesabı, fonksiyonel bir programlama dili olan LISP'in temelini oluşturmuştur.

C# dilindeki Lambda ifadeleri **anonim yöntemler** (**anonymous method**) gibi kullanılır. Lamda ifadelerinde girdi değerlerin veri tipini belirtmenize gerek yoktur. Bu da kullanımı daha esnek hale getirir. Lamda ifadesi iki bölüme ayrılır, sol taraf girdidir, sonrasında lamda işleci (**=>**) bulunur. Sağ taraf ise **ifadedir** (**expression**).

Talimatların (**statement**), yapılaması gerekeni kısa ve net olarak belirten **mantıksal cümlelerdir** (**logical sequences**) olduğu önceden anlatılmıştı. Talimatlar **anahtar kelimeler** (**keyword**) içerirler ve noktalı virgül karakteri ile biter. Bazı talimatlar, **ifade** (**expression**) olarak adlandırılır. Yalnızca sabit, değişken ve **işleç** (**operator**) içeren talimatlara ifade adı verilir. Bu tanımlardan hareketle lamda ifadesi ve lamda talimatı olmak üzere iki tür lamda vardır.

Lamda İfadeleri

Lamda ifadesi (**lambda expression**) aşağıdaki gibi yazılır;

```
girdi => ifade;
```

Aşağıda buna ilişkin bir örnek verilmiştir;

```
delegate int ikiIntParametreliTmsilci(int input1, int input2);
ikiIntParametreliTmsilci ikiTamsayıyıTopla = (x,y) => x + y;
ikiIntParametreliTmsilci ikiTamsayıyıÇarp = (x,y) => x * y;
```

delegate kelimesini kullanmadan geri değer döndürmeyen **Action** ve geri değer döndüren **Func** kelimeleri ile lamda ifadeleri tanımlayabiliriz. Aşağıda **delegate** olmadan kullanılan lamda ifadelerine yer verilmiştir;

```
// See https://aka.ms/new-console-template for more information
Action line = () => Console.WriteLine("-----"); //Parametresiz
line();
//Action delegate'leri, geriye değer döndürmezler!

Func<double, double> cube = x => x * x * x; //Tek Parametreli
Console.WriteLine(cube(3));
//Func delegate'leri, geriye değer döndürürler!
/* Yukarıdaki kodun açılımı
double cube(double x) {
    return x * x * x;
}
*/

Func<int, double, double> kat = (x,y) => x * y; //İki Parametreli
Console.WriteLine(kat(3,6.0));
/* Yukarıdaki kodun açılımı
double kat(int x, double y) {
    return x * y;
}
*/

//Veri tipi içeren parametreli
Func<int, string, bool> MetinKarakterSayısıVerilendenBüyükMü = (int x, string s)
    => s.Length > x;
Console.WriteLine(MetinKarakterSayısıVerilendenBüyükMü(3, "Merhaba"));

//Parametreler Göz Ardı Edildi
Func<int, int, int> constant = (_, _) => 61;
Console.WriteLine(constant(3, 5)); // Output: 61
```

Lamda Talimatları

Lamda talimatı (**lambda tatement**) aşağıdaki gibi tanımlanır;

```
girdi => { talimatlar };
```

Aşağıda örneği vermiştir;

```
// See https://aka.ms/new-console-template for more information
Func<int, string> doubleThenAddElevenThenQuote = i => {
    var doubled = 2 * i;
    var addedEleven = 11 + doubled;
    return $"{addedEleven}";
}
doubleThenAddElevenThenQuote(5); // "'21'"
```

Temsilci Başlatmada Lamda İfadeleri

Temsilciler (*delegate*) yöntemleri dolaylı çağırmanın yoludur. C dilindeki fonksiyon göstericileri gibidir ancak ondan daha yeteneklidirler. Temsilcilere ilk değer verme yani başlatmada lamda ifadeleri kullanılır;

```
// See https://aka.ms/new-console-template for more information
public delegate int intGirdiAlanveIntDönerBirTemsilci(int input);
class Program
{
    static void Main(string[] args)
    {
        intGirdiAlanveIntDönerBirTemsilci ikiİleÇarp = x => x * 2;
        intGirdiAlanveIntDönerBirTemsilci karesiniAl = x => x * x;
        intGirdiAlanveIntDönerBirTemsilci küpünüAl = x => x * x * x;
        int sayi = 10;
        Console.WriteLine("10*2 = " + ikiİleÇarp(10)); // 20
        Console.WriteLine("sayi^2 = " + karesiniAl(sayi)); //100
        Console.WriteLine("sayi^3 = " + küpünüAl(sayi)); //1000
    }
}
```

Anonim Değişkenlerle Lamda İfadeleri

Bir lamda ifadesini tanımlarken **var** kelimesi ile temsilci tipi otomatik olarak çıkarılır. Aslında genel olarak bir ifadenin sonucunda ortaya çıkan veri tipinin çıkarımı otomatik olması isteniyorsa anonim değişkenler kullanılır.

```
var IncrementBy = (int source, int increment = 1) => source + increment;
Console.WriteLine(IncrementBy(5)); // 6
Console.WriteLine(IncrementBy(5, 2)); // 7
```

Ayrıca, açıkça yazılmış bir parametre listesindeki son parametre olarak, parametre dizileri veya koleksiyonları olan lambda ifadelerini de tanımlanabilir;

```
// See https://aka.ms/new-console-template for more information
var sum = (params IEnumerable<int> values) =>
{
    int sum = 0;
    foreach (var value in values)
        sum += value;
    return sum;
};
var empty = sum();
Console.WriteLine(empty); // 0
var sequence = new[] { 1, 2, 3, 4, 5 };
var total = sum(sequence);
Console.WriteLine(total); // 15
```

Anonim Yöntemler

Anonim yöntemler (**anonymous method**) bir kod bloğunu **temsilci** (**delegate**) parametresi olarak geçirmek için bir teknik sağlar. Bunlar bir gövdeye sahip ancak adı olmayan yöntemlerdir ve Lamda ifadeleri ile tanımlanırlar.

```
// See https://aka.ms/new-console-template for more information
delegate int IntOp(int lhs, int rhs);
class Program
{
    static void Main(string[] args)
    {
        // C# 2.0 definition
        IntOp add = delegate(int lhs, int rhs)
        {
            return lhs + rhs;
        };
        // C# 3.0 definition: Uzun
        IntOp mul = (lhs, rhs) =>
        {
            return lhs * rhs;
        };
        // C# 3.0 definition: Kısa
        IntOp sub = (lhs, rhs) => lhs - rhs;
        // Her bir anonim yöntemi çağıralım:
        Console.WriteLine("2 + 3 = " + add(2, 3));
        Console.WriteLine("2 * 3 = " + mul(2, 3));
        Console.WriteLine("2 - 3 = " + sub(2, 3));
    }
}
```

Olaylar

Nesneler, **olaylardan** (**event**) etkilenirler ve olaylar karşısında tepki verir ya da **davranış** (**behavior**) gösterirler. Olayı **yayan** (**publish**) nesneye diğer tüm nesneler tepki vermezler. Bu nedenle tepki verecek nesneler, davranış göstereceği olaya önceden **abone** (**subscribe**) olurlar. Abone olunan olay gerçekleştiği zaman **teпки** (**handler**) verirler.

Aşağıda bir saat nesnesinin tık sesi olayına tepki veren dinleyici nesnesi örneğine yer verilmiştir;

```
// See https://aka.ms/new-console-template for more information
/* "Kişi" sınıfında bir olay tanımlanacak.
 * "YaşDeğiştİ" olayına nasıl yöntemlerle abone olunacağına
 * "OlayOluncaAboneOlunacakYöntem" delegate ile karar verilir. */
public delegate void OlayOluncaAboneOlunacakYöntem(Kişi s);
public class Kişi
{
    /* "Kişi" sınıfında bir olay tanımlanır.
     * "YaşDeğiştİ" olayı, "yaş" değiştiğinde tetiklenecek olan bir olaydır. */
    public event OlayOluncaAboneOlunacakYöntem yaşDeğiştİ;
    public string İsim { get; set; }
    private int yaş;
    public int Yaş
    {
        get { return yaş; }
        set
        {
            if (yaş != value)
            {
                yaş = value;
                if (yaşDeğiştİ != null)

```

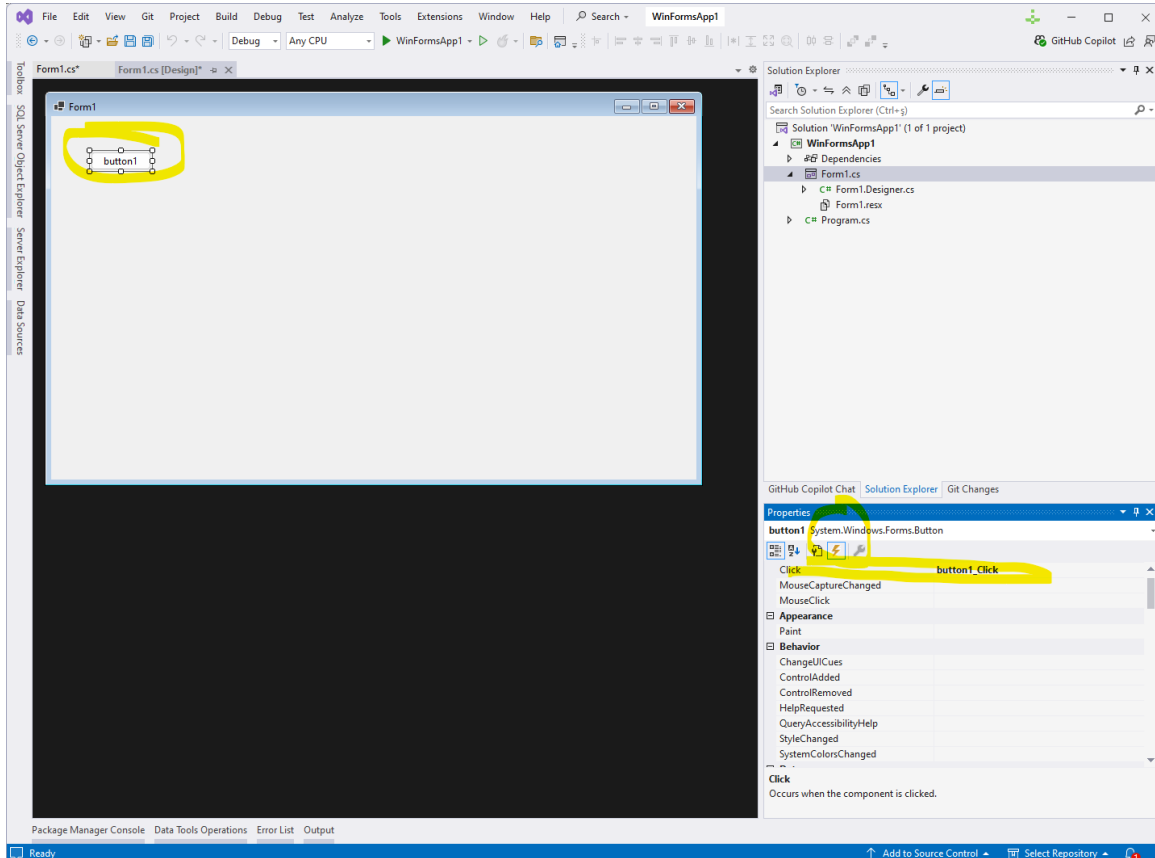
```

        {
            yaşDeğiřti(this); //Olayı tetikle!
        }
    }
}

public class Dinleyici
{
    /* "Dinleyici" sınıfından imal edilen nesneler,
    * "Kiři" sınıfından imal edilen nesnelere abone olabilir. */
    public void AboneOl(Kiři k)
    {
        k.yaşDeğiřti += this.kiřininYaşıDeğiřinceNeYapılacaksaYap;
    }
    /*Kiřide olay gerekleřtiğinde vereceėi tepkiye iliřkin davranıř tanımlanmalıdır. */
    private void kiřininYaşıDeğiřinceNeYapılacaksaYap(Kiři s)
    {
        Console.WriteLine("Kiřinin Yaşı Deėiřti!");
    }
}

internal class Program
{
    static void Main(string[] args)
    {
        Kiři kiři = new Kiři();
        Dinleyici dinleyici = new Dinleyici();
        dinleyici.AboneOl(kiři);
        kiři.Yaş = 20;
        kiři.Yaş = 30;
    }
}

```



řekil 25. Örnek Bir Windows Forms Uygulaması

C# dilinde ön tanımlı olarak olaylar aşağıdaki gibi tanımlıdır;

```
public event EventHandler<EventArgsT> EventName;
```

`<EventArgsT>` için jenerik başlığını inceleyebilirsiniz. Olaylara tepki verecek yöntemler ve abonelik aşağıdaki gibi tanımlanır;

```
public void HandlerName(object sender, EventArgsT args) { /* Handler logic */ }
EventName += HandlerName; // Olaya Abone Olma
```

Eğer Windows Forms uygulaması geliştiriyorsanız form tasarımında aşağıdaki kod verilebilir. Şekil 25'te verilen formda **Click** olayına çift tıklandığında IDE tarafından otomatik olarak bir tepki yöntemi hazırlanır ve bu yöntem bu olarak abone edilir.

```
namespace WinFormsApp1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        private void button1_Click(object sender, EventArgs e)
        {
        }
        private void Form1_Load(object sender, EventArgs e)
        {
        }
    }
}
```

Eğer bir olay olduğunda parametreler ile çalışacak isek EventArgs sınıfından türeyen bir sınıf tanımlamalıyız. Yukarıda verilen Kişi sınıfını C# için öntanımlı olan **EventArgs** ile aşağıdaki gibi yazabiliriz;

```
// See https://aka.ms/new-console-template for more information
public class YaşDeğişmeOlayıArgs : EventArgs
{
    public YaşDeğişmeOlayıArgs(int mevcutYaş, int yeniYaş)
    {
        this.MevcuyYaş = mevcutYaş;
        this.YeniYaş = yeniYaş;
    }
    public int MevcuyYaş { get; private set; }
    public int YeniYaş { get; private set; }
}

public delegate void YaşDeğişinceAboneOlunacakYöntem(object sender,
                                                    YaşDeğişmeOlayıArgs args);

public class Kişi
{
    public event YaşDeğişinceAboneOlunacakYöntem yaşDeğişt;
    public string İsim { get; set; }
    private int yaş;
    public int Yaş
    {
        get { return yaş; }
        set
        {
            if (yaş != value)
            {
```

```

        if (yaşDeğiştirdi != null)
        {
            YaşDeğişmeOlayıArgs args = new YaşDeğişmeOlayıArgs(yaş, value);
            yaşDeğiştirdi(this, args); //Olayı Tetikle!
        }
        yaş = value;
    }
}

public class Dinleyici
{
    /* "Dinleyici" sınıfından imal edilen nesneler,
    * "Kişi" sınıfından imal edilen nesnelere abone olabilir. */
    public void AboneOl(Kişi k)
    {
        k.yaşDeğiştirdi += this.kişininYaşıDeğişinceYap;
    }
    public void kişininYaşıDeğişinceYap(object sender, YaşDeğişmeOlayıArgs args)
    {
        Console.WriteLine($"{args.MevcuyYaş} olan Kişinin Yaşı "+
            $"{args.YeniYaş} olarak Değişti!");
    }
}

internal class Program
{
    static void Main(string[] args)
    {
        Kişi kişi = new Kişi();
        Dinleyici dinleyici = new Dinleyici();
        dinleyici.AboneOl(kişi);
        kişi.Yaş = 20;
        kişi.Yaş = 30;
    }
}

```

Olaylara Tepki Vermek İçin Lamda İfadeleri

Lamda ifadeleri **olaylara** (event) **tepki** (handler) vermek için kullanılabilir. Bu durumda tepki gösterilecek olaya **abone** (subscribe) olduktan sonra abonelikten ayrılma söz konusu olmaz.

```

// See https://aka.ms/new-console-template for more information
public class YaşDeğişmeOlayıArgs : EventArgs
{
    public YaşDeğişmeOlayıArgs(int mevcutyaş, int yeniyaş)
    {
        this.MevcuyYaş = mevcutyaş;
        this.YeniYaş = yeniyaş;
    }
    public int MevcuyYaş { get; private set; }
    public int YeniYaş { get; private set; }
}

public delegate void YaşDeğişinceAboneOlunacakYöntem(object sender, YaşDeğişmeOlayıArgs args);
public class Kişi
{
    public Kişi()
    {
        yaş = 0;
        İsim = "İsimsiz";
        yaşDeğiştirdi = null;
    }
    public event YaşDeğişinceAboneOlunacakYöntem yaşDeğiştirdi;
}

```

```

public string İsim { get; set; }
private int yaş;
public int Yaş
{
    get { return yaş; }
    set
    {
        if (yaş != value)
        {
            if (yaşDeğiştirdi != null)
            {
                YaşDeğişmeOlayıArgs args = new YaşDeğişmeOlayıArgs(yaş, value);
                yaşDeğiştirdi(this, args);
            }
            yaş = value;
        }
    }
}
}
public class Dinleyici
{
    /* "Dinleyici" sınıfından imal edilen nesneler,
    * "Kişi" sınıfından imal edilen nesnelere abone olabilir. */
    public void AboneOl(Kişi k)
    {
        k.yaşDeğiştirdi += this.kişininYaşıDeğişinceYap;
    }
    public void kişininYaşıDeğişinceYap(object sender, YaşDeğişmeOlayıArgs args)
    {
        Console.WriteLine($"{args.MevcuyYaş} olan Kişinin Yaşı "+
            $"{args.YeniYaş} olarak Değişti!");
    }
}
internal class Program
{
    static void Main(string[] args)
    {
        Kişi kişi = new Kişi();
        Dinleyici dinleyici = new Dinleyici();
        dinleyici.AboneOl(kişi);
        kişi.Yaş = 20;
        kişi.Yaş = 30;
        Console.WriteLine("----- ");
        YaşDeğişinceAboneOlunacakYöntem tepki = (sender, args)
            => Console.WriteLine("Kişinin Yaşı Değişemez!!");

        kişi.yaşDeğiştirdi += tepki;
        kişi.yaşDeğiştirdi += (sender, args)
            => Console.WriteLine("Kişinin Yaşı Savcılık Emriyle Değişebilir!");

        kişi.Yaş = 40;
    }
}

```

Burada lamda ifadesiyle tepki olarak kodlanan yöntem `kişi.yaşDeğiştirdi -= tepki`; talimatı yazılarak abonelikten çıkılabilir. Ancak onun altındaki anonim yöntem için abonelikten vazgeçilemez!

Genişleme Yöntemleri

Genişleme yöntemleri (*extension method*), C# 3.0 ile hayatımıza girmiştir. Genişletme yöntemleri, yeni bir türetilmiş tip oluşturmada, yeniden derlemeden veya orijinal türü başka şekilde değiştirmeden mevcut türlere davranışı genişletir ve ekler. Özellikle geliştirmek istediğiniz bir tipin kaynağını değiştiremediğinizde faydalıdırlar. Genişletme yöntemleri `int`, `char`, `string` gibi sistem tipleri, üçüncü

tarafarla kütüphanelerde tanımlanan tipler ve **class** gibi kendiniz tanımladığınız tipler için oluşturulabilir. Genişletme yöntemi orijinal tipin bir üye yöntemiymiş gibi çağrılabilir.

Bir genişletme yönteminin ilk parametresi her zaman **this** anahtar sözcüğünden önce gelmeli, ardından genişlettiğiniz nesnenin "geçerli" örneğine başvurmak için tanımlayıcı gelmelidir;

```
// See https://aka.ms/new-console-template for more information
static class StringExtensions
{
    public static string Shorten(this string text, int length)
    {
        return text.Substring(0, length);
    }
}
class Program
{
    static void Main()
    {
        // aşağıdaki atama String.ToUpper() yöntemini çağırır.
        var myString = "Hello World!".ToUpper();
        /* aşağıdaki yöntem Extensions sınıfının StringExtensions.Shorten()
        * yöntemini çağırır */
        var newString = myString.Shorten(5);
        // Aşağıda ise genişletilmiş yöntem çağırılmıştır.
        var newString2 = StringExtensions.Shorten(myString, 5);
    }
}
```

Bir sınıfın kaynak kodu mevcut değilse veya mevcut sınıfta değişiklik yapma iznimiz yoksa, genişleme yöntemlerinin mevcut sınıfa yeni yöntemler ekleyerek bir sınıfın işlevselliğini genişletmek için bir yaklaşım olarak kullanılabilir. Hatırlamanız gereken en önemli nokta, genişleme yöntemlerinin statik bir sınıfın statik yöntemleri olduğu, ancak genişletilmiş tipteki örnek yöntemlermiş gibi çağrılacaklarıdır.

İsim Uzayları

İsim uzayları (**namespace**) kodun düzenli, okunabilir ve yönetilebilir olmasını sağlamak için kullanılan bir yapıdır. Özellikle büyük projelerde isim çakışmalarını önlemek ve sınıfları mantıksal olarak gruplamak için kullanılır.

İsim uzayları, sınıf, ara yüz, yapı (**struct**) ve numaralandırma (**enum**) gibi kod bileşenlerini mantıksal gruplara ayırmak için kullanılır. Temel amacı isim çakışmalarını önlemek ve kodu düzenlemektir.

namespace anahtar kelimesi, bir kod içerisinde yer alan bileşenlerin nasıl düzenlendiğini anlamamıza yardımcı olur. Fiziksel bir klasöre benzer şekilde bileşenleri tasnif etmemizi sağlar. Bir isim uzayı iç içe istenildiği kadar isim uzayı barındırabilir.

İsim uzayının adını belirtmeden içindeki bileşenleri kullanabilmek için **using** anahtar kelimesi kullanılır. Aksi durumda sınıflarda olduğu gibi üyelere nokta işlecisi erişilebilir.

```
// See https://aka.ms/new-console-template for more information
using İsimUzayı;
Kullanıcı kullanıcı1 = new Kullanıcı
{
    Ad = "Ahmet",
    Soyad = "Yılmaz",
    Tip = KullanıcıTipi.ADMIN
};
Kullanıcı kullanıcı2 = new Kullanıcı
{
    Ad = "Mehmet",
    Soyad = "Demir",
    Tip = KullanıcıTipi.RAPORTÖR
}
```

```

};
İsimUzayı2.Kullanıcı kullanıcı3 = new İsimUzayı2.Kullanıcı
{
    Ad = "Ayşe",
    Soyad = "Kara"
};

namespace İsimUzayı
{
    enum KullanıcıTipi { ADMIN, RAPORTÖR, İZLEYİCİ }
    class Kullanıcı
    {
        public string Ad { get; set; }
        public string Soyad { get; set; }
        public KullanıcıTipi Tip { get; set; }
    }
}

namespace İsimUzayı2
{
    class Kullanıcı
    {
        public string Ad { get; set; }
        public string Soyad { get; set; }
    }
}

```

İsim uzayları nokta ile ayrılan fiziksel klasörlere benzer;

```

// See https://aka.ms/new-console-template for more information
İsimUzayı.KullanıcıArayüzü.Kullanıcı.Kullanıcı kullanıcı1 =
    new İsimUzayı.KullanıcıArayüzü.Kullanıcı.Kullanıcı
    {
        Ad = "Ahmet",
        Soyad = "Yılmaz",
        Tip = İsimUzayı.KullanıcıArayüzü.Kullanıcı.KullanıcıTipi.ADMIN
    };
kullanıcı1.Ad = "Ali"; // Değişiklik yapıldı

namespace İsimUzayı.KullanıcıArayüzü.Kullanıcı
{
    enum KullanıcıTipi { ADMIN, RAPORTÖR, İZLEYİCİ }
    class Kullanıcı
    {
        public string Ad { get; set; }
        public string Soyad { get; set; }
        public KullanıcıTipi Tip { get; set; }
    }
}

```

İsim uzaylarını **using** kelimesi ile referans göstererek içerdiği bileşenleri kullanabiliriz;

```

// See https://aka.ms/new-console-template for more information
using System.Text;

var sb = new StringBuilder(); //<- var sb = new System.Text.StringBuilder(); yerine

```

İsim uzaylarına takma ad vererek de kullanabiliriz;

```

using st = System.Text;

var sb = new st.StringBuilder(); //<- var sb = new System.Text.StringBuilder(); yerine

```